

Final Project

December 4, 2024

1 CS-GY 6053 Foundations of Data Science (F24)

2 Final Project

Group Members: David Hong(sh8348), Yanka Sikder(ys4780), Farhen Shefa(fas6488)

2.1 1. Identify a question/estimand

- **Question: Does GDP influence life expectancy?**
- **Description of the data:** The dataset we are using is the Gapminder dataset (<https://www.gapminder.org/data/>) by Jennifer Bryan which contains data on life expectancy and GDP per capita by country and year. It contains 1704 observations from 2022 and contains 6 variables: country, continent, year, lifeExp, pop, and gdpPercap. Since we are only allowed to use three variables, we will be using pop - population, gdpPercap - GDP per capita (US\$, inflation-adjusted), lifeExp - Life expectancy at birth, in years. We will be utilizing the data from this csv file: <https://github.com/NickCHK/causaldata/blob/main/Python/causaldata/gapminder/gapminder.csv>

2.2 2. Describe a scientific/causal model

We imported necessary libraries at the beginning of the notebook. Also, We used the same code settings from our lectures and assignments.

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import arviz as az
import pymc as pm
import seaborn as sns
from scipy import stats
import networkx as nx
import warnings
warnings.filterwarnings('ignore', category=FutureWarning)

# Set random seed and ensure plots are displayed in the notebook
rnd_seed = 42
np.random.seed(rnd_seed)
```

```
%matplotlib inline
%config Inline.figure_format = 'retina'

# sets default credible interval used by arviz
az.rcParams["stats.hdi_prob"] = 0.89
```

2.2.1 2.1 Causal Model Description

We used a DAG to visualize the potential causal relationships between variables:

1. **Main Relationship of Interest:**

- GDP per capita $\rightarrow$ Life Expectancy

2. **Potential Confounder:**

- Population affects both GDP per capita and Life Expectancy

3. **Variables:**

- **Treatment:** GDP (gdpPercap) - Gross Domestic Product per capita, (US\$, inflation-adjusted)
- **Outcome:** Life Expectancy (lifeExp) - Life expectancy at birth, in years
- **Confounder:** Population (pop) - Total population of the country

2.2.2 2.2 Causal Inference Strategy

1. **Control for Confounding**

- Included Population in our model to address confounding

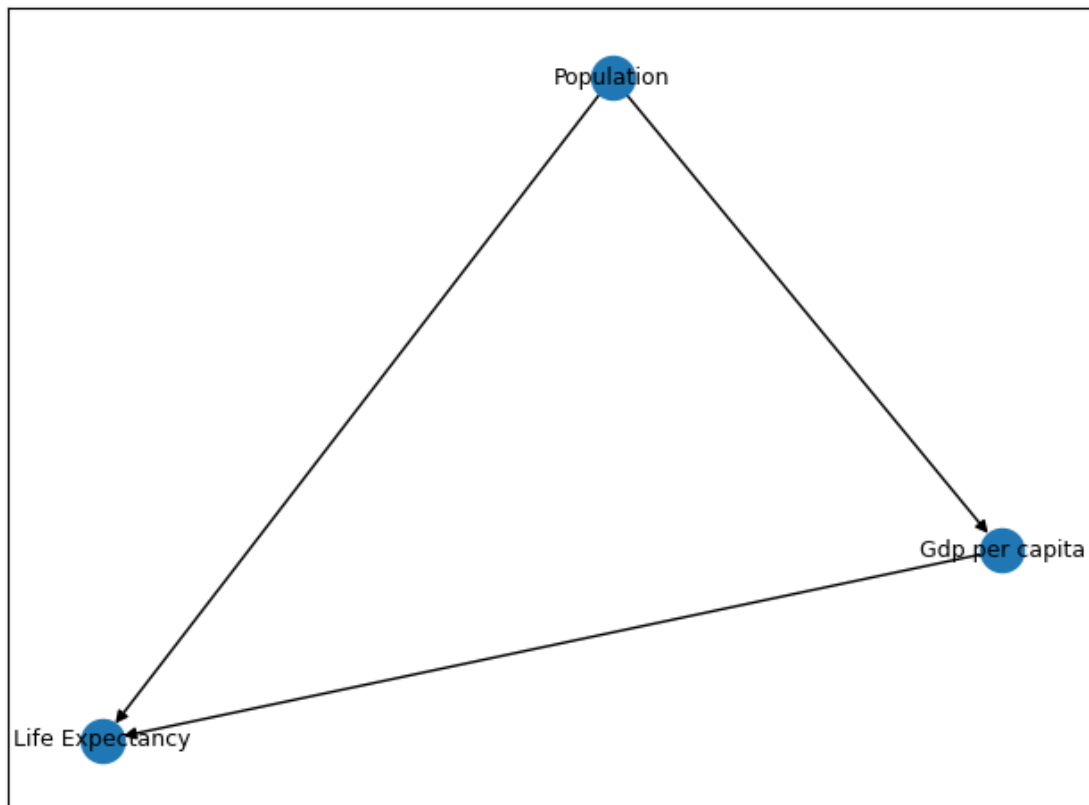
2. **Effect Analysis**

- Total Effect: Overall relationship between GDP and Life Expectancy
- Direct Effect: GDP's effect on Life Expectancy controlling for Population

2.2.3 2.3 Causal Assumptions

- We assume that GDP affects Life Expectancy (not vice versa) and that controlling for Population is sufficient for causal inference.

```
[2]: graph = nx.DiGraph()
graph.add_edges_from([("Population", "Gdp per capita"), ("Gdp per capita", "Life_
↪ Expectancy"), ("Population", "Life Expectancy")])
nx.draw_networkx(graph, arrows=True, font_size = 9)
plt.tight_layout()
```



2.3 3. Define a corresponding statistical model

```
[3]: # Load and name data
gapminder_url = "https://raw.githubusercontent.com/NickCH-K/causaldata/main/
↳Python/causaldata/gapminder/gapminder.csv"
df_gapminder = pd.read_csv(gapminder_url)

# Select only the variables we need for analysis
df = df_gapminder[['lifeExp', 'gdpPercap', 'pop']]
```

```
[4]: # Check basic information
print("Dataset Info:")
print(df.info())

print("\nFirst 5 rows:")
print(df.head())

# Check missing values
print("\nMissing values:")
print(df.isnull().sum())
```

```

print("\nBasic statistics:")
print(df.describe())

# Visualize distributions of variables
plt.figure(figsize=(15, 5))

plt.subplot(131)
sns.histplot(df['lifeExp'])
plt.title('Life Expectancy Distribution')

plt.subplot(132)
sns.histplot(df['gdpPercap'])
plt.title('GDP Distribution')

plt.subplot(133)
sns.histplot(df['pop'])
plt.title('Population Distribution')

plt.tight_layout()
plt.show()

# Visualize relationships between variables
plt.figure(figsize=(15, 5))

plt.subplot(131)
sns.scatterplot(data=df, x='gdpPercap', y='lifeExp')
plt.title('GDP vs Life Expectancy')

plt.subplot(132)
sns.scatterplot(data=df, x='pop', y='lifeExp')
plt.title('Population vs Life Expectancy')

plt.subplot(133)
sns.scatterplot(data=df, x='pop', y='gdpPercap')
plt.title('Population vs GDP')

plt.tight_layout()
plt.show()

```

Dataset Info:

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1704 entries, 0 to 1703
Data columns (total 3 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   lifeExp     1704 non-null   float64

```

```

1   gdpPercap  1704 non-null   float64
2   pop        1704 non-null   int64
dtypes: float64(2), int64(1)
memory usage: 40.1 KB
None

```

First 5 rows:

	lifeExp	gdpPercap	pop
0	28.801	779.445314	8425333
1	30.332	820.853030	9240934
2	31.997	853.100710	10267083
3	34.020	836.197138	11537966
4	36.088	739.981106	13079460

Missing values:

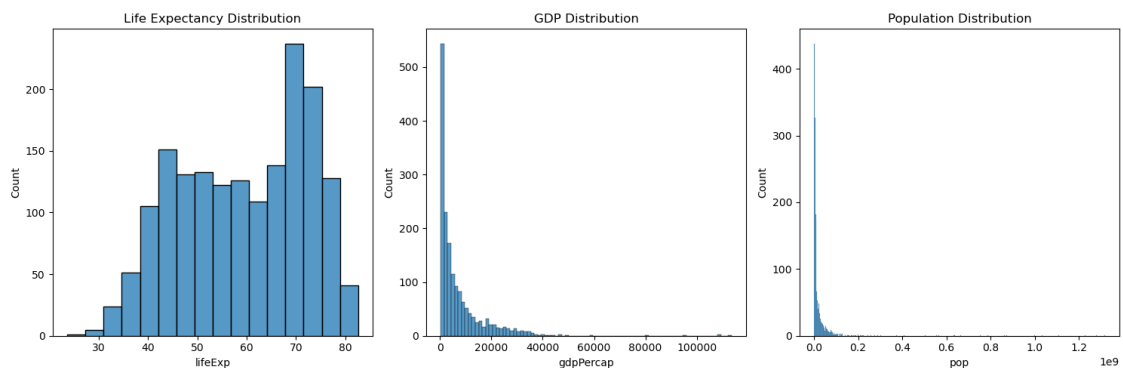
```

lifeExp      0
gdpPercap    0
pop          0
dtype: int64

```

Basic statistics:

	lifeExp	gdpPercap	pop
count	1704.000000	1704.000000	1.704000e+03
mean	59.474439	7215.327081	2.960121e+07
std	12.917107	9857.454543	1.061579e+08
min	23.599000	241.165876	6.001100e+04
25%	48.198000	1202.060309	2.793664e+06
50%	60.712500	3531.846988	7.023596e+06
75%	70.845500	9325.462346	1.958522e+07
max	82.603000	113523.132900	1.318683e+09





We conducted exploratory data analysis to examine distributions, identify missing values, and understand potential outliers in our dataset.

Univariate Analysis - Life expectancy: Shows a relatively symmetric, normal distribution, suggesting standard scaling would be appropriate.

- GDP: Shows a positively skewed distribution, with many countries having low GDP and few having extremely high values. The distribution follows relatively a logarithmic series distribution.
- Population: Similarly shows a positively skewed distribution, with many countries having small populations and few having large populations. The distribution follows relatively a logarithmic series distribution.

Bivariate Analysis

We implemented bivariate analysis to visualize the relationships between life expectancy, GDP, and population. Since we are comparing features that have different values, ranges, and distribution; the shapes of the bivariate graphs are difficult to analyze. Due to the influence of the logarithmic distribution, all three scatterplots follow that shape, and are clustered on near the origin. They all also have obvious and large outliers which indicates the shape is not the best reflection of the data.

While the life expectancy variable shows a relatively normal distribution and can be effectively standardized using the same scaling method we used in previous lectures and assignments, the other two variables (GDP and population) have many extreme values. Therefore, applying a different type of scaling method would be more effective for these variables.

```
[5]: # Distribution Analysis: Comparing Original, Standardized, and Log-transformed
      ↪ data
      # to identify optimal scaling methods for each variable

      def standardize(series):
          return (series - series.mean()) / series.std()

      fig, axes = plt.subplots(3, 3, figsize=(15, 12))

      variables = ['gdpPerCap', 'lifeExp', 'pop']
```

```

titles = ['Original', 'Standardized', 'Log']

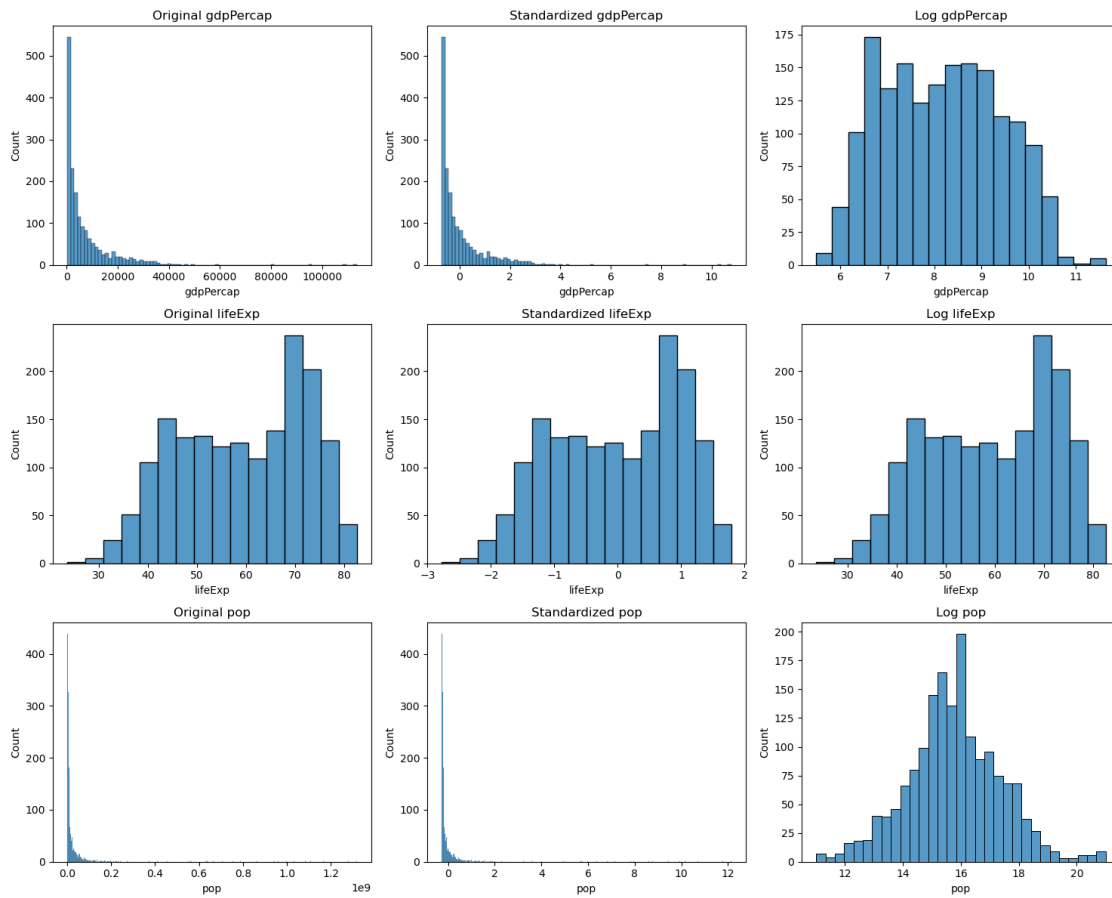
for i, var in enumerate(variables):
    # Original
    sns.histplot(df[var], ax=axes[i,0])
    axes[i,0].set_title(f'Original {var}')

    # Standardized
    sns.histplot(standardize(df[var]), ax=axes[i,1])
    axes[i,1].set_title(f'Standardized {var}')

    # Log transformation
    if var in ['gdpPercap', 'pop']:
        sns.histplot(np.log(df[var]), ax=axes[i,2])
    else:
        sns.histplot(df[var], ax=axes[i,2])
    axes[i,2].set_title(f'Log {var}')

plt.tight_layout()
plt.show()

```



Since the data are very spread out, we implemented three scaling methods: standardization, log-scaling, log-scaling followed by standardization.

GDP

- The distribution using standardization is similar to the original distribution – heavily positively skewed, but values are scaled to have a mean of 0 and a standard deviation of 1.
- The distribution using log-scaling spreads the lower range, resulting in a distribution closer to a normal distribution, which improves interpretability.

Life expectancy

- The distribution using standardization is the same as the original distribution – slightly negatively skewed, but values are scaled to have a mean of 0 and a standard deviation of 1.
- The distribution using log-scaling does not significantly change the distribution since the values are already distributed reasonably.

Population

- The distribution population using standardization is similar to the original distribution – heavily positively skewed, but values are scaled to have a mean of 0 and a standard deviation of 1.
- The distribution of population using log-scaling compresses the values, resulting in a distribution closer to a normal distribution, which is easier to interpret.

```
[6]: # Scaling the data:
# - Direct standardization for life expectancy (normal-like distribution)
# - Log transformation followed by standardization for GDP and population ↵
    ↵(skewed distributions)

lifeExp_std = standardize(df['lifeExp'])
gdp_std = standardize(np.log(df['gdpPercap']))
pop_std = standardize(np.log(df['pop']))

# Create a DataFrame for scaled variables
df_scaled = pd.DataFrame({
    'lifeExp_std': lifeExp_std,
    'gdp_std': gdp_std,
    'pop_std': pop_std
})
```

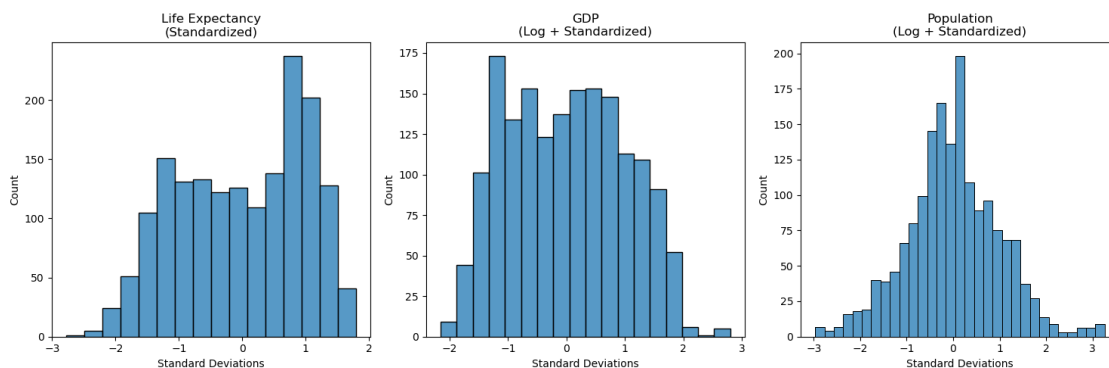
```
[7]: # Visualize distributions of transformed variables
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# Life expectancy (standardized)
sns.histplot(lifeExp_std, ax=axes[0])
axes[0].set_title('Life Expectancy\n(Standardized)')
axes[0].set_xlabel('Standard Deviations')
```

```
# GDP (log-transformed and standardized)
sns.histplot(gdp_std, ax=axes[1])
axes[1].set_title('GDP\n(Log + Standardized)')
axes[1].set_xlabel('Standard Deviations')

# Population (log-transformed and standardized)
sns.histplot(pop_std, ax=axes[2])
axes[2].set_title('Population\n(Log + Standardized)')
axes[2].set_xlabel('Standard Deviations')

plt.tight_layout()
plt.show()
```



For GDP and population, we applied log-scaling followed by standardization to ensure a consistent scaling and improve interpretability since the only log-scaled distributions covered a large range of values. However, for life expectancy, we did not apply log-scaling but we did apply standardization since the distribution was fairly normally distributed.

Thus, for our data, we log-scaled and standardized GDP and population to normalize their distribution, while only standardizing life expectancy.

```
[8]: # Visualization of Relationships between Standardized Variables
fig, axes = plt.subplots(1, 3, figsize=(20, 5))

# Plot 1: Standardized log GDP vs Standardized Life Expectancy
sns.scatterplot(data=df_scaled, x='gdp_std', y='lifeExp_std', alpha=0.5,
               ↪ax=axes[0])
axes[0].set_title('GDP (log, standardized) vs Life Expectancy (standardized)')
axes[0].set_xlabel('Log GDP per capita (standardized)')
axes[0].set_ylabel('Life Expectancy (standardized)')

# Plot 2: Standardized log Population vs Standardized Life Expectancy
sns.scatterplot(data=df_scaled, x='pop_std', y='lifeExp_std', alpha=0.5,
               ↪ax=axes[1])
```

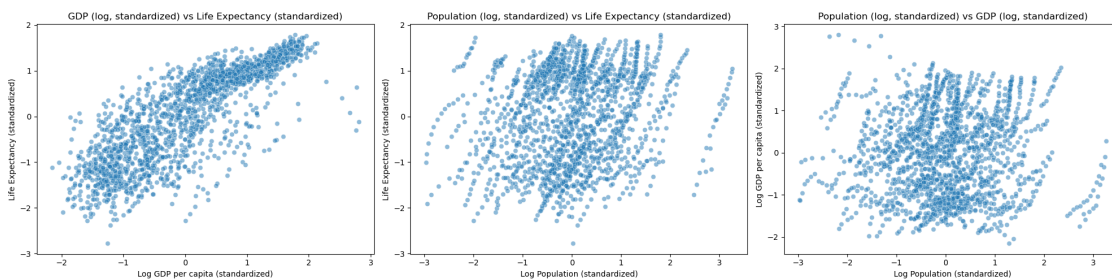
```

axes[1].set_title('Population (log, standardized) vs Life Expectancy_
↳(standardized)')
axes[1].set_xlabel('Log Population (standardized)')
axes[1].set_ylabel('Life Expectancy (standardized)')

# Plot 3: Standardized log Population vs Standardized log GDP
sns.scatterplot(data=df_scaled, x='pop_std', y='gdp_std', alpha=0.5, ax=axes[2])
axes[2].set_title('Population (log, standardized) vs GDP (log, standardized)')
axes[2].set_xlabel('Log Population (standardized)')
axes[2].set_ylabel('Log GDP per capita (standardized)')

plt.tight_layout()
plt.show()

```



Bivariate Analysis

We then implemented bivariate analysis to visualize and understand the relationships between life expectancy, GDP, and population after standardizing and/or log scaling.

GDP (log, standardized) vs. Life expectancy (standardized)

- The scatterplot shows a strong positive linear trend with few outliers.
- Makes sense as higher GDP is associated with high life expectancy.

Population (log, standardized) vs. Life expectancy (standardized) - The scatterplot is more scattered and clustered in the middle. - Does not show a linear relationship.

Population (log, standardized) vs. GDP (log, standardized) - The scatterplot is also more scattered, and clustered in the middle. - The relationship is complex and nonlinear. - There are various outliers that are affecting the distribution, making its independent effect difficult to isolate. - These results are evidence that the population is a confounder as it is difficult to identify its effect on life expectancy and GDP.

The simultaneous variation of priors (alpha, beta_gdp, beta_pop) is preferred over sequential optimization for several key reasons:

1. **Efficiency:** Testing combinations requires fewer iterations (5 runs) compared to sequential testing (15 runs)
2. **Real-world Representation:** Variables like GDP, population, and life expectancy don't operate in isolation, making simultaneous prior variation more realistic

Justification of Prior Settings for Prior Predictive Simulation:

We designed our prior settings from a Normal distribution based on three key principles:

1. Baseline Effect (α):
 - Centered at 0 across all settings
 - Uncertainty increases from 0.2 to 1.0
 - Allows for varying levels of baseline life expectancy
2. GDP Effect (β_{gdp}):
 - Starts with strong positive effect (0.7) in conservative settings
 - Gradually decreases to moderate effect (0.3) in weakly informative settings
 - Uncertainty (standard deviation) increases from 0.1 to 0.5
 - Reflects our belief in positive relationship between GDP and life expectancy
3. Population Effect (β_{pop}):
 - Maintains neutral mean (0) across all settings
 - Uncertainty increases from 0.1 to 0.5
 - Reflects our uncertainty about population's impact on life expectancy

```
[9]: # Prior predictive simulation with 30 samples for each setting
n = 30
# Create sequences of GDP and population values
gdp_seq = np.linspace(gdp_std.min(), gdp_std.max())
pop_seq = np.linspace(pop_std.min(), pop_std.max())

# Different prior settings to compare
prior_settings = {
    'Very Conservative': {
        'alpha': (0, 0.2),
        'beta_gdp': (0.7, 0.1), # Strong positive GDP effect
        'beta_pop': (0, 0.1)    # Neutral population effect, small uncertainty
    },
    'Conservative': {
        'alpha': (0, 0.4),
        'beta_gdp': (0.6, 0.2),
        'beta_pop': (0, 0.2)
    },
    'Balanced': {
        'alpha': (0, 0.6),
        'beta_gdp': (0.5, 0.3),
        'beta_pop': (0, 0.3)
    },
    'Informative': {
        'alpha': (0, 0.8),
        'beta_gdp': (0.4, 0.4),
        'beta_pop': (0, 0.4)
    },
    'Weakly informative': {
        'alpha': (0, 1),
```

```

        'beta_gdp': (0.3, 0.5), # Weak positive GDP effect
        'beta_pop': (0, 0.5)    # Neutral population effect, large uncertainty
    }
}

# Create subplots: 5 rows (one for each prior setting) and 2 columns (GDP and
↳Population effects)
fig, axes = plt.subplots(len(prior_settings), 2, figsize=(15,
↳5*len(prior_settings)))

# Iterate through each prior setting
for i, (prior_name, priors) in enumerate(prior_settings.items()):
    # Generate prior samples
    alpha = stats.norm.rvs(priors['alpha'][0], priors['alpha'][1], size=n)
    beta_gdp = stats.norm.rvs(priors['beta_gdp'][0], priors['beta_gdp'][1],
↳size=n)
    beta_pop = stats.norm.rvs(priors['beta_pop'][0], priors['beta_pop'][1],
↳size=n)

    # GDP effect plot
    pop_mean = np.mean(pop_std) # Fix population at its mean value
    for j in range(n):
        # Calculate predicted life expectancy for each GDP value
        mu_prior_gdp = alpha[j] + beta_gdp[j] * gdp_seq + beta_pop[j] * pop_mean
        # Plot one possible relationship line
        axes[i,0].plot(gdp_seq, mu_prior_gdp, color='blue', alpha=0.2)

    axes[i,0].scatter(gdp_std, lifeExp_std, alpha=0.3, color='teal') # Plot
↳actual data points
    axes[i,0].set_xlabel("GDP per capita (log-transformed, standardized)")
    axes[i,0].set_ylabel("Life Expectancy (standardized)")
    axes[i,0].set_title(f'{prior_name} Priors\nGDP Effect')
    axes[i,0].grid(True, alpha=0.3)

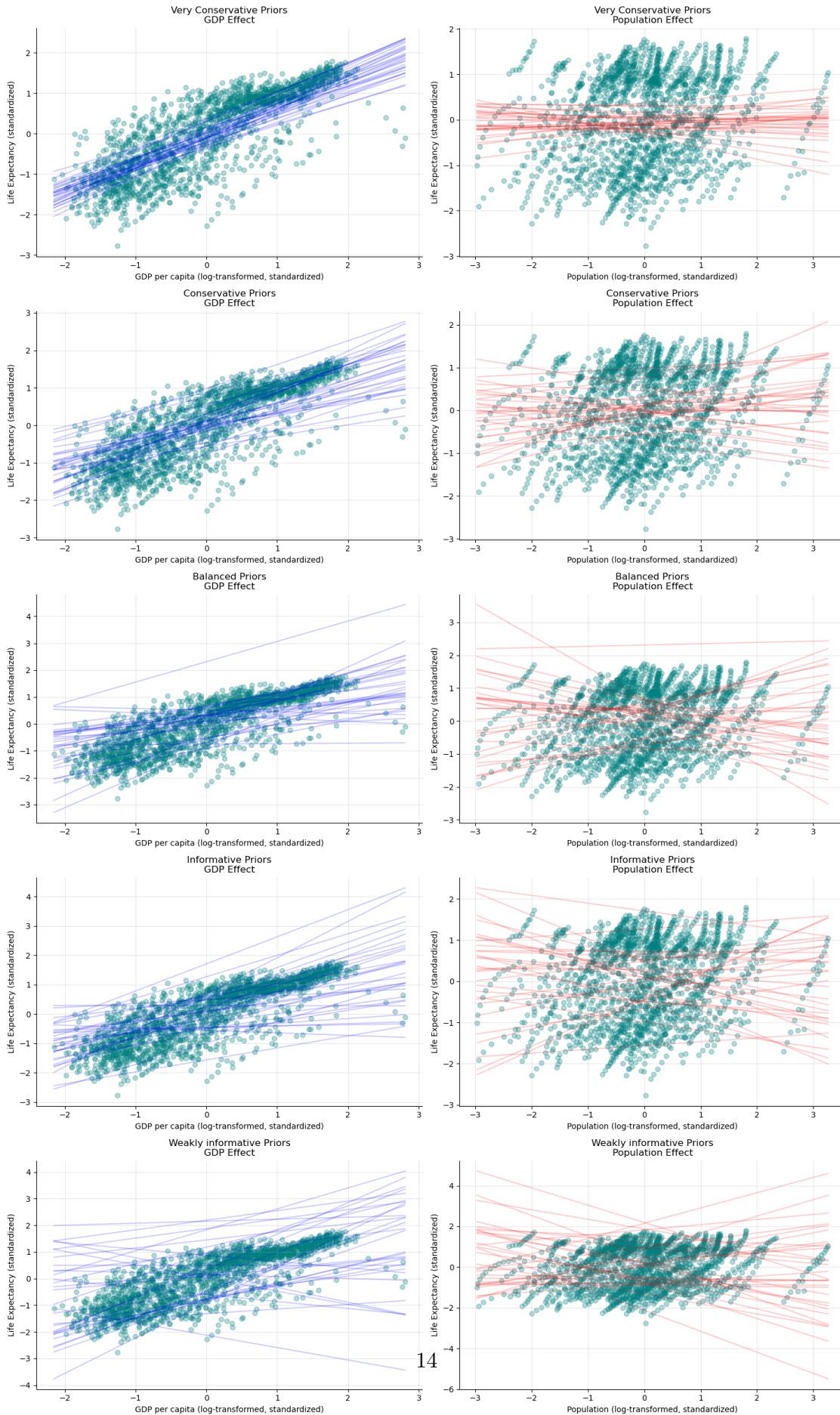
    # Population effect plot
    gdp_mean = np.mean(gdp_std) # Fix GDP at its mean value
    for j in range(n):
        # Calculate predicted life expectancy for each population value
        mu_prior_pop = alpha[j] + beta_gdp[j] * gdp_mean + beta_pop[j] * pop_seq
        # Plot one possible relationship line
        axes[i,1].plot(pop_seq, mu_prior_pop, color='red', alpha=0.2)

    axes[i,1].scatter(pop_std, lifeExp_std, alpha=0.3, color='teal') # Plot
↳actual data points
    axes[i,1].set_xlabel("Population (log-transformed, standardized)")
    axes[i,1].set_ylabel("Life Expectancy (standardized)")

```

```
axes[i,1].set_title(f'{prior_name} Priors\nPopulation Effect')
axes[i,1].grid(True, alpha=0.3)

plt.tight_layout()
sns.despine()
plt.show()
```



Very Conservative: $\sim N(0, 0.2)$, $\text{__gdp} \sim N(0.7, 0.1)$, $\text{__pop} \sim N(0, 0.1)$

- Regression lines for GDP are clustered tightly, implying strong prior assumptions that GDP has a significant and consistent positive effect.
- Almost flat and tightly clustered regression lines for population, indicating minimal or no population effect.

Conservative: $\sim N(0, 0.4)$, $\text{__gdp} \sim N(0.6, 0.2)$, $\text{__pop} \sim N(0, 0.2)$ & **Balanced:** $\sim N(0, 0.6)$, $\text{__gdp} \sim N(0.5, 0.3)$, $\text{__pop} \sim N(0, 0.3)$ - Conservative and balanced priors show increased variability in the slopes, allowing more flexibility in capturing different GDP effects. - Slopes for population vary more, implying increased uncertainty in the effect of population.

Informative: $\sim N(0, 0.8)$, $\text{__gdp} \sim N(0.4, 0.4)$, $\text{__pop} \sim N(0, 0.4)$ & **Weakly Informative:** $\sim N(0, 1)$, $\text{__gdp} \sim N(0.3, 0.5)$, $\text{__pop} \sim N(0, 0.5)$ - Informative and weakly informative priors show highly varied regression lines, which capture a broad range of plausible GDP relationships. - Regression lines for population show a lot more variation, allowing for both positive and negative population effect

We chose the balanced setting ($\sim N(0, 0.6)$, $\text{__gdp} \sim N(0.5, 0.3)$, $\text{__pop} \sim N(0, 0.3)$) because it: - Maintains reasonable assumption about GDP's positive effect - Allows sufficient uncertainty to capture real-world variability, and preventing from overfitting to noise and outliers, enhancing predictive performance - Allows model to adapt to data without being too influenced by the prior assumptions - Takes neutral stance on population effects - Shows good alignment with observed data in prior predictive checks - Resulting posterior estimates more likely to be interpretable and credible,

Once the priors for the model were decided, a few of the basic models discussed in the lecture such as intercept-only models and then predictor-inclusive models were created to best understand the effect of the treatment and confounder variable on the outcome variable.

Model 1: Intercept-only model

The Intercept model is a baseline model where it assumes no effect of GDP or population on life expectancy. It evaluated how well the intercept alone explains life expectancy.

Likelihood: $y_i \sim \text{Normal}(\mu_i, \sigma^2)$ - $\mu_i =$

Priors: $\sigma^2 \sim \text{Normal}(0, 0.6)$ - $\sigma \sim \text{Exponential}(1)$

Model 2: Total Effect model (GDP only)

The second Predictor-Inclusive Model introduces the GDP variable and examines the overall effect of GDP on Life expectancy.

Likelihood: $y_i \sim \text{Normal}(\mu_i, \sigma^2)$ - $\mu_i = \alpha + \text{__gdp} * \text{GDP}_i$

Priors: $\sigma^2 \sim \text{Normal}(0, 0.6)$ - $\text{__gdp} \sim \text{Normal}(0.5, 0.3)$ - $\sigma \sim \text{Exponential}(1)$

Model 3: Direct Effect model (GDP + Population)

The third Predictor-Inclusive Model introduces the confounding variable population while still estimating the direct effect of GDP on life expectancy.

Likelihood: $-y_i \sim \text{Normal}(_i,) - _i = _ + _gdp * GDP_i + _pop * POP_i$

Priors: $- \sim \text{Normal}(0, 0.6) - _gdp \sim \text{Normal}(0.5, 0.3) - _pop \sim \text{Normal}(0, 0.3) - \sim \text{Exponential}(1)$

```
[10]: # 1. Intercept-only model
with pm.Model() as intercept_model:
    alpha = pm.Normal("alpha", mu=0, sigma=0.6)
    sigma = pm.Exponential("sigma", 1)
    mu = pm.Deterministic("mu", alpha)
    y = pm.Normal("y", mu=mu, sigma=sigma, observed=lifeExp_std)
    idata_intercept = pm.sample(return_inferencedata=True,
    ↪ idata_kwargs={"log_likelihood": True})

# 2. Total Effect model (GDP only)
with pm.Model() as total_effect_model:
    alpha = pm.Normal("alpha", mu=0, sigma=0.6)
    beta_gdp = pm.Normal("beta_gdp", mu=0.5, sigma=0.3)
    sigma = pm.Exponential("sigma", 1)
    mu = pm.Deterministic("mu", alpha + beta_gdp * gdp_std)
    y = pm.Normal("y", mu=mu, sigma=sigma, observed=lifeExp_std)
    idata_total = pm.sample(return_inferencedata=True,
    ↪ idata_kwargs={"log_likelihood": True})

# 3. Direct Effect model (GDP + Population)
with pm.Model() as direct_effect_model:
    alpha = pm.Normal("alpha", mu=0, sigma=0.6)
    beta_gdp = pm.Normal("beta_gdp", mu=0.5, sigma=0.3)
    beta_pop = pm.Normal("beta_pop", mu=0, sigma=0.3)
    sigma = pm.Exponential("sigma", 1)
    mu = pm.Deterministic("mu", alpha + beta_gdp * gdp_std + beta_pop * pop_std)
    y = pm.Normal("y", mu=mu, sigma=sigma, observed=lifeExp_std)
    idata_direct = pm.sample(return_inferencedata=True,
    ↪ idata_kwargs={"log_likelihood": True})
```

Auto-assigning NUTS sampler...

Initializing NUTS using jitter+adapt_diag...

Sequential sampling (2 chains in 1 job)

NUTS: [alpha, sigma]

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

Sampling 2 chains for 1_000 tune and 1_000 draw iterations (2_000 + 2_000 draws total) took 3 seconds.

We recommend running at least 4 chains for robust computation of convergence diagnostics

```

Auto-assigning NUTS sampler...
Initializing NUTS using jitter+adapt_diag...
Sequential sampling (2 chains in 1 job)
NUTS: [alpha, beta_gdp, sigma]

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

Sampling 2 chains for 1_000 tune and 1_000 draw iterations (2_000 + 2_000 draws
total) took 7 seconds.
We recommend running at least 4 chains for robust computation of convergence
diagnostics
Auto-assigning NUTS sampler...
Initializing NUTS using jitter+adapt_diag...
Sequential sampling (2 chains in 1 job)
NUTS: [alpha, beta_gdp, beta_pop, sigma]

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

Sampling 2 chains for 1_000 tune and 1_000 draw iterations (2_000 + 2_000 draws
total) took 6 seconds.
We recommend running at least 4 chains for robust computation of convergence
diagnostics

```

```

[11]: # Create dictionary of models
model_dict = {
    "Intercept-only": idata_intercept,
    "Total Effect": idata_total,
    "Direct Effect": idata_direct
}

# Compare using both PSIS-LOO and WAIC
compare_df_psis = az.compare(compare_dict=model_dict, ic="loo", scale="deviance")
compare_df_waic = az.compare(compare_dict=model_dict, ic="waic", scale="deviance")

# Visualize PSIS-LOO comparison
print("PSIS-LOO Comparison:")
display(compare_df_psis)
_ = az.plot_compare(compare_df_psis)
plt.title("Model Comparison (PSIS-LOO)")
plt.show()

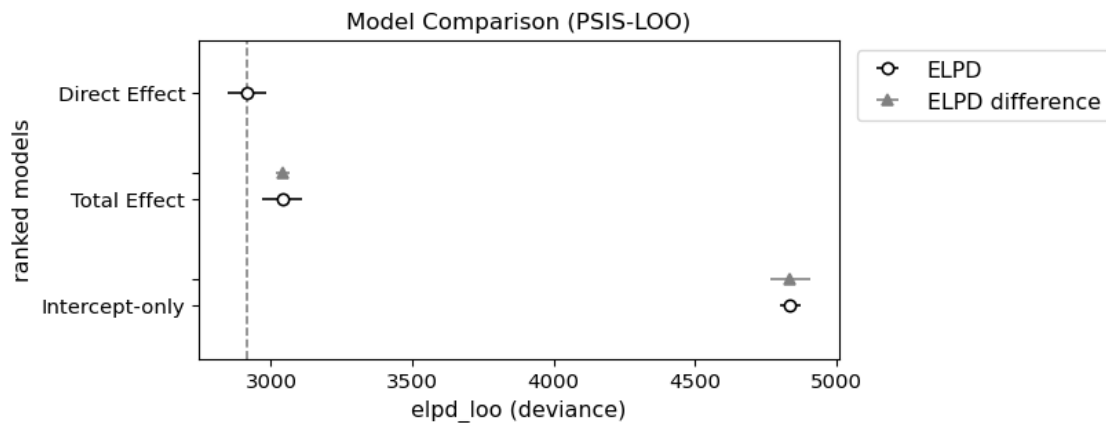
```

```
# Visualize WAIC comparison
print("WAIC Comparison:")
display(compare_df_waic)
_ = az.plot_compare(compare_df_waic)
plt.title("Model Comparison (WAIC)")
plt.show()
```

PSIS-LOO Comparison:

	rank	elpd_loo	p_loo	elpd_diff	weight	se	\
Direct Effect	0	2915.188963	4.497426	0.000000	0.976288	67.463142	
Total Effect	1	3040.970334	3.256459	125.781371	0.002422	71.296790	
Intercept-only	2	4838.135771	1.419490	1922.946808	0.021290	38.555711	

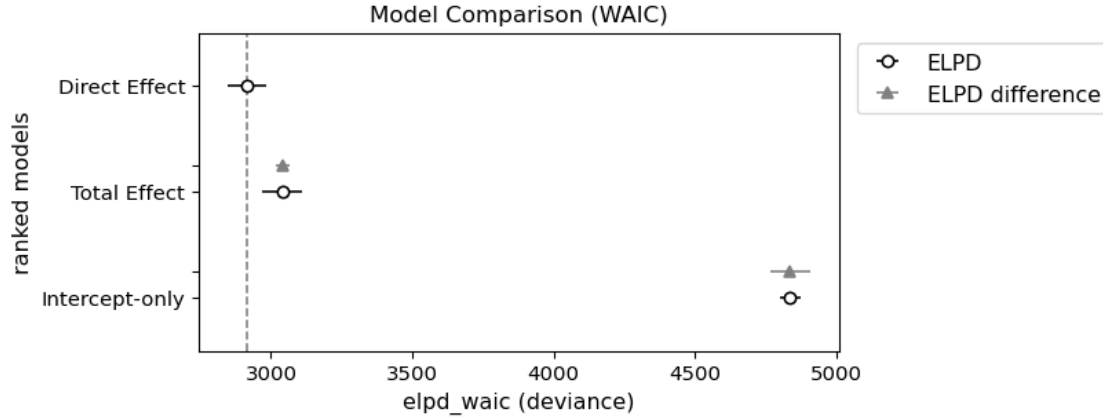
	dse	warning	scale
Direct Effect	0.000000	False	deviance
Total Effect	23.394457	False	deviance
Intercept-only	70.501582	False	deviance



WAIC Comparison:

	rank	elpd_waic	p_waic	elpd_diff	weight	se	\
Direct Effect	0	2915.168972	4.487430	0.000000	0.976311	67.462081	
Total Effect	1	3040.951631	3.247107	125.782659	0.002401	71.295223	
Intercept-only	2	4838.129512	1.416361	1922.960540	0.021288	38.555550	

	dse	warning	scale
Direct Effect	0.000000	False	deviance
Total Effect	23.394206	False	deviance
Intercept-only	70.500588	False	deviance



Our model comparison analysis shows that the Direct Effect model outperformed both the Total Effect model (GDP only) and the Intercept-only model. This is evidenced by better PSIS-LOO and WAIC scores, since the `elpd_loo` and `elpd_waic` value of direct effect are the lowest. This suggests that including population as a control variable might slightly enhance our model's ability to explain variations in life expectancy.

After analyzing and confirming that our casual model assumptions are correct, we will test 3 different models: Linear, Quadratic, and Cubic to determine the best statistical model to utilize for our dataset. Using higher-order terms seen in quadratic and cubic models allows us to capture non-linear relationships within our dataset. This will also allow us to identify if we are overfitting with a higher-order model or if our dataset requires a higher-order model to handle the complexities within it. We can define the best model that captures the relationships to correctly balance accuracy with interpretability of the complexity within the dataset.

In addition, since we utilized log scaling on the GDP and population values we have to utilize different priors for higher-order terms are used for these key reasons:

1. **Model Stability:** Higher-order terms can make models unstable, so tighter priors (smaller sigma) help control this
2. **Prevent Overfitting:** Stricter priors on higher-order terms act as regularization
3. **Mathematical Properties:** Higher powers grow exponentially ($x^1=10$, $x^2=100$, $x^3=1000$), requiring tighter constraints

By handling the priors, we correctly account for and handle the confounding variable (population) so that the statistical models can prioritize on predicting the effect of GDP on life expectancy.

Model 1: Linear model

This model assumes that the relationship between life expectancy, GDP, and population is linear. It is the simplest model and we predict that will perform the worst.

Likelihood: $y_i \sim \text{Normal}(\mu_i, \sigma)$ - $\mu_i = \beta_0 + \beta_{\text{gdp}} * \text{GDP}_i + \beta_{\text{pop}} * \text{POP}_i$

Priors: $\beta_0 \sim \text{Normal}(0, 0.6)$ - $\beta_{\text{gdp}} \sim \text{Normal}(0.5, 0.3)$ - $\beta_{\text{pop}} \sim \text{Normal}(0, 0.3)$ - $\sigma \sim \text{Exponential}(1)$

Model 2: Quadratic model

This model introduces the second-order terms to capture the non-linear relationships between GDP and population on life expectancy.

Likelihood: $-y_i \sim \text{Normal}(_i,) - _i = + _gdp * GDP_i + _gdp2 * GDP_i^2 + _pop * POP_i + _pop2 * POP_i^2$

Priors: $- \sim \text{Normal}(0, 0.6) - _gdp \sim \text{Normal}(0.5, 0.3) - _gdp2 \sim \text{Normal}(0, 0.2) - _pop \sim \text{Normal}(0, 0.3) - _pop2 \sim \text{Normal}(0, 0.2) - \sim \text{Exponential}(1)$

Model 3: Cubic model

This is the most complex model we are testing as it introduces a third-order term to capture nuanced relationships between the variables. The issue with this model is that it risks overfitting, especially in cases with limited data which is not the case here.

Likelihood: $-y_i \sim \text{Normal}(_i,) - _i = + _gdp * GDP_i + _gdp2 * GDP_i^2 + _gdp3 * GDP_i^3 + _pop * POP_i + _pop2 * POP_i^2 + _pop3 * POP_i^3$

Priors: $- \sim \text{Normal}(0, 0.6) - _gdp \sim \text{Normal}(0.5, 0.3) - _gdp2 \sim \text{Normal}(0, 0.2) - _gdp3 \sim \text{Normal}(0, 0.1) - _pop \sim \text{Normal}(0, 0.3) - _pop2 \sim \text{Normal}(0, 0.2) - _pop3 \sim \text{Normal}(0, 0.1) - \sim \text{Exponential}(1)$

```
[12]: # Different polynomial models for comparison
with pm.Model() as linear_model:
    alpha = pm.Normal("alpha", mu=0, sigma=0.6)
    beta_gdp = pm.Normal("beta_gdp", mu=0.5, sigma=0.3)
    beta_pop = pm.Normal("beta_pop", mu=0, sigma=0.3)
    sigma = pm.Exponential("sigma", 1)
    mu = pm.Deterministic("mu", alpha + beta_gdp * gdp_std + beta_pop * pop_std)
    y = pm.Normal("y", mu=mu, sigma=sigma, observed=lifeExp_std)
    idata_linear = pm.sample(return_inferencedata=True,
↳ idata_kwargs={"log_likelihood": True})

with pm.Model() as quadratic_model:
    alpha = pm.Normal("alpha", mu=0, sigma=0.6)
    beta_gdp = pm.Normal("beta_gdp", mu=0.5, sigma=0.3)
    beta_gdp2 = pm.Normal("beta_gdp2", mu=0, sigma=0.2) # quadratic term for
↳ GDP
    beta_pop = pm.Normal("beta_pop", mu=0, sigma=0.3)
    beta_pop2 = pm.Normal("beta_pop2", mu=0, sigma=0.2) # quadratic term for
↳ population
    sigma = pm.Exponential("sigma", 1)
    mu = pm.Deterministic("mu", alpha +
        beta_gdp * gdp_std + beta_gdp2 * gdp_std**2 +
        beta_pop * pop_std + beta_pop2 * pop_std**2)
    y = pm.Normal("y", mu=mu, sigma=sigma, observed=lifeExp_std)
    idata_quad = pm.sample(return_inferencedata=True,
↳ idata_kwargs={"log_likelihood": True})

with pm.Model() as cubic_model:
```

```

alpha = pm.Normal("alpha", mu=0, sigma=0.6)
beta_gdp = pm.Normal("beta_gdp", mu=0.5, sigma=0.3)
beta_gdp2 = pm.Normal("beta_gdp2", mu=0, sigma=0.2)
beta_gdp3 = pm.Normal("beta_gdp3", mu=0, sigma=0.1) # cubic term for GDP
beta_pop = pm.Normal("beta_pop", mu=0, sigma=0.3)
beta_pop2 = pm.Normal("beta_pop2", mu=0, sigma=0.2)
beta_pop3 = pm.Normal("beta_pop3", mu=0, sigma=0.1) # cubic term for
↳population
sigma = pm.Exponential("sigma", 1)
mu = pm.Deterministic("mu", alpha +
                      beta_gdp * gdp_std + beta_gdp2 * gdp_std**2 +
↳beta_gdp3 * gdp_std**3 +
                      beta_pop * pop_std + beta_pop2 * pop_std**2 +
↳beta_pop3 * pop_std**3)
y = pm.Normal("y", mu=mu, sigma=sigma, observed=lifeExp_std)
idata_cubic = pm.sample(return_inferencedata=True,
↳idata_kwargs={"log_likelihood": True})

```

Auto-assigning NUTS sampler...

Initializing NUTS using jitter+adapt_diag...

Sequential sampling (2 chains in 1 job)

NUTS: [alpha, beta_gdp, beta_pop, sigma]

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

Sampling 2 chains for 1_000 tune and 1_000 draw iterations (2_000 + 2_000 draws total) took 5 seconds.

We recommend running at least 4 chains for robust computation of convergence diagnostics

Auto-assigning NUTS sampler...

Initializing NUTS using jitter+adapt_diag...

Sequential sampling (2 chains in 1 job)

NUTS: [alpha, beta_gdp, beta_gdp2, beta_pop, beta_pop2, sigma]

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

Sampling 2 chains for 1_000 tune and 1_000 draw iterations (2_000 + 2_000 draws total) took 9 seconds.

We recommend running at least 4 chains for robust computation of convergence diagnostics

Auto-assigning NUTS sampler...
 Initializing NUTS using jitter+adapt_diag...
 Sequential sampling (2 chains in 1 job)
 NUTS: [alpha, beta_gdp, beta_gdp2, beta_gdp3, beta_pop, beta_pop2, beta_pop3, sigma]
 <IPython.core.display.HTML object>
 <IPython.core.display.HTML object>
 <IPython.core.display.HTML object>
 <IPython.core.display.HTML object>
 Sampling 2 chains for 1_000 tune and 1_000 draw iterations (2_000 + 2_000 draws total) took 15 seconds.
 We recommend running at least 4 chains for robust computation of convergence diagnostics

```
[13]: # Create dictionary of models
model_dict = {
    "Linear": idata_linear,
    "Quadratic": idata_quad,
    "Cubic": idata_cubic,
}

# Compare using both PSIS-LOO and WAIC
compare_df_psis = az.compare(compare_dict=model_dict, ic="loo",
                             ↪scale="deviance")
compare_df_waic = az.compare(compare_dict=model_dict, ic="waic",
                             ↪scale="deviance")

# Visualize PSIS-LOO comparison
print("PSIS-LOO Comparison:")
display(compare_df_psis)
_ = az.plot_compare(compare_df_psis)
plt.title("Model Comparison (PSIS-LOO)")
plt.show()

# Visualize WAIC comparison
print("WAIC Comparison:")
display(compare_df_waic)
_ = az.plot_compare(compare_df_waic)
plt.title("Model Comparison (WAIC)")
plt.show()
```

PSIS-LOO Comparison:

/opt/conda/envs/fnds/lib/python3.9/site-packages/arviz/stats/stats.py:1645:
 UserWarning: For one or more samples the posterior variance of the log
 predictive densities exceeds 0.4. This could be indication of WAIC starting to

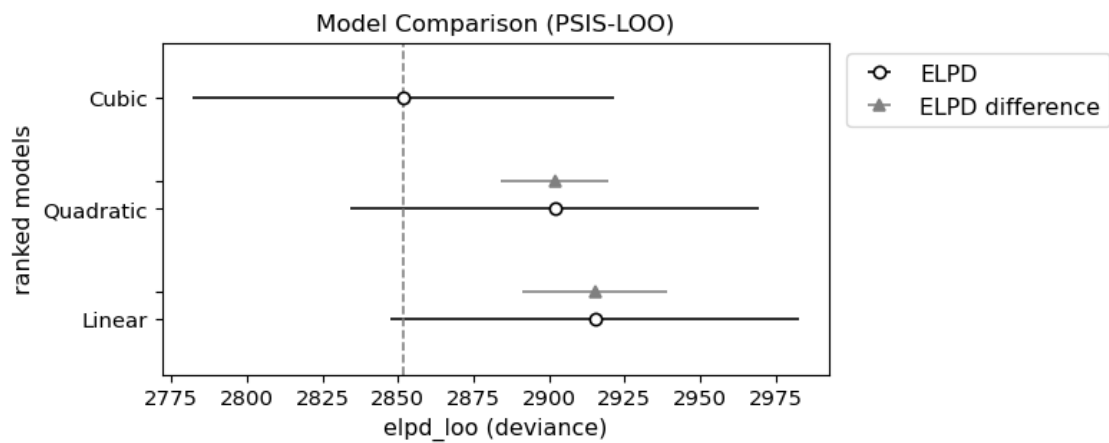
fail.

See <http://arxiv.org/abs/1507.04544> for details

```
warnings.warn(
```

	rank	elpd_loo	p_loo	elpd_diff	weight	se \
Cubic	0	2851.633793	8.598586	0.000000	0.88309	69.667626
Quadratic	1	2901.928676	7.029353	50.294883	0.00000	67.675508
Linear	2	2915.135356	4.471469	63.501563	0.11691	67.436438

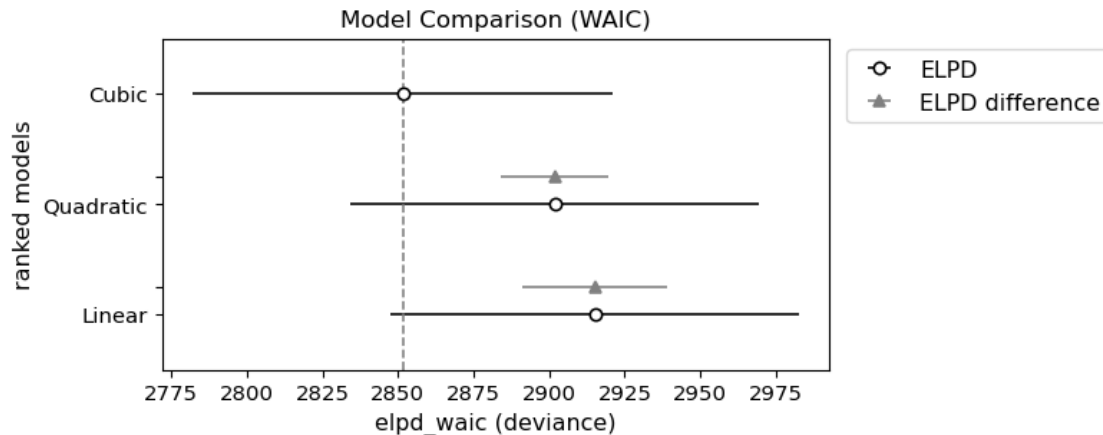
	dse	warning	scale
Cubic	0.000000	False	deviance
Quadratic	17.906305	False	deviance
Linear	23.760860	False	deviance



WAIC Comparison:

	rank	elpd_waic	p_waic	elpd_diff	weight	se \
Cubic	0	2851.573539	8.568459	0.000000	8.834860e-01	69.664860
Quadratic	1	2901.871216	7.000623	50.297678	5.230135e-16	67.671263
Linear	2	2915.119697	4.463639	63.546159	1.165140e-01	67.435842

	dse	warning	scale
Cubic	0.000000	False	deviance
Quadratic	17.899143	True	deviance
Linear	23.763974	False	deviance



Our comparison of polynomial models demonstrated that the Cubic model provided the best fit to the data, outperforming both the Linear and Quadratic models based on PSIS-LOO and WAIC criteria. It outperformed the Quadratic model by ~50 points and the Linear model by ~63 which indicates that it is a moderate increase in performance for PSIS. The second best model based on the PSIS and WAIC results is the Quadratic model which indicates that there are complex relationships between the variables in the dataset that these higher-order models can handle. The Cubic model suggests that the relationship between GDP, population, and life expectancy exhibits non-linear patterns that are better captured by higher-order polynomial terms. Thus, our statistical and posterior model will be a cubic model to best simulate and model observed data.

2.4 4. Validate the model on simulated data with known parameter values

```
[14]: # 1. Generate simulation data
n_samples = 1000
sim_alpha = 0
sim_beta_gdp = 0.5      # prior mean from our actual model
sim_beta_gdp2 = 0.0     # prior mean for quadratic term
sim_beta_gdp3 = 0.0     # prior mean for cubic term
sim_beta_pop = 0.0      # prior mean from our actual model
sim_beta_pop2 = 0.0     # prior mean for quadratic term
sim_beta_pop3 = 0.0     # prior mean for cubic term
sim_sigma = 1

# Generate covariates
sim_gdp = np.random.normal(0, 1, n_samples)
sim_pop = np.random.normal(0, 1, n_samples)

# Generate outcome variable with cubic terms
sim_life_exp = (sim_alpha +
                sim_beta_gdp * sim_gdp + sim_beta_gdp2 * sim_gdp**2 +
                ↪sim_beta_gdp3 * sim_gdp**3 +
```

```

        sim_beta_pop * sim_pop + sim_beta_pop2 * sim_pop**2 +
↪sim_beta_pop3 * sim_pop**3 +
        np.random.normal(0, sim_sigma, n_samples))

# 2. Fit cubic model to simulated data
with pm.Model() as sim_model:
    alpha = pm.Normal("alpha", mu=0, sigma=0.6)
    beta_gdp = pm.Normal("beta_gdp", mu=0.5, sigma=0.3)
    beta_gdp2 = pm.Normal("beta_gdp2", mu=0, sigma=0.2) # quadratic term for
↪GDP
    beta_gdp3 = pm.Normal("beta_gdp3", mu=0, sigma=0.1) # cubic term for GDP
    beta_pop = pm.Normal("beta_pop", mu=0, sigma=0.3)
    beta_pop2 = pm.Normal("beta_pop2", mu=0, sigma=0.2) # quadratic term for
↪population
    beta_pop3 = pm.Normal("beta_pop3", mu=0, sigma=0.1) # cubic term for
↪population
    sigma = pm.Exponential("sigma", 1)

# Cubic model
mu = pm.Deterministic("mu", alpha +
    beta_gdp * sim_gdp + beta_gdp2 * sim_gdp**2 + beta_gdp3 * sim_gdp**3 +
    beta_pop * sim_pop + beta_pop2 * sim_pop**2 + beta_pop3 * sim_pop**3)

# Likelihood
y = pm.Normal("y", mu=mu, sigma=sigma, observed=sim_life_exp)

# Sampling
idata_sim = pm.sample(500, return_inferencedata=True)

# 3. Results
az.summary(idata_sim)

```

Auto-assigning NUTS sampler...

Initializing NUTS using jitter+adapt_diag...

Sequential sampling (2 chains in 1 job)

NUTS: [alpha, beta_gdp, beta_gdp2, beta_gdp3, beta_pop, beta_pop2, beta_pop3, sigma]

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

Sampling 2 chains for 1_000 tune and 500 draw iterations (2_000 + 1_000 draws total) took 10 seconds.

We recommend running at least 4 chains for robust computation of convergence diagnostics

```
[14]:
```

	mean	sd	hdi_5.5%	hdi_94.5%	mcse_mean	mcse_sd	ess_bulk	\
alpha	0.025	0.047	-0.050	0.100	0.002	0.001	938.0	
beta_gdp	0.553	0.048	0.477	0.630	0.002	0.001	786.0	
beta_gdp2	0.007	0.024	-0.028	0.048	0.001	0.001	945.0	
beta_gdp3	-0.001	0.013	-0.020	0.019	0.000	0.000	773.0	
beta_pop	-0.022	0.048	-0.099	0.053	0.002	0.001	719.0	
...	...	...	...	...	...	...	...	
mu[995]	0.294	0.052	0.208	0.371	0.002	0.001	838.0	
mu[996]	-0.373	0.101	-0.544	-0.229	0.003	0.002	1046.0	
mu[997]	0.685	0.064	0.576	0.783	0.002	0.001	1031.0	
mu[998]	0.326	0.053	0.237	0.405	0.002	0.001	827.0	
mu[999]	-0.561	0.061	-0.660	-0.467	0.002	0.002	806.0	

	ess_tail	r_hat
alpha	845.0	1.0
beta_gdp	669.0	1.0
beta_gdp2	817.0	1.0
beta_gdp3	685.0	1.0
beta_pop	466.0	1.0
...	...	...
mu[995]	722.0	1.0
mu[996]	951.0	1.0
mu[997]	887.0	1.0
mu[998]	850.0	1.0
mu[999]	850.0	1.0

[1008 rows x 9 columns]

```
[15]: # summary of specific parameters
az.summary(idata_sim,
           var_names=['alpha',
                     'beta_gdp', 'beta_gdp2', 'beta_gdp3',
                     'beta_pop', 'beta_pop2', 'beta_pop3',
                     'sigma'])
```

```
[15]:
```

	mean	sd	hdi_5.5%	hdi_94.5%	mcse_mean	mcse_sd	ess_bulk	\
alpha	0.025	0.047	-0.050	0.100	0.002	0.001	938.0	
beta_gdp	0.553	0.048	0.477	0.630	0.002	0.001	786.0	
beta_gdp2	0.007	0.024	-0.028	0.048	0.001	0.001	945.0	
beta_gdp3	-0.001	0.013	-0.020	0.019	0.000	0.000	773.0	
beta_pop	-0.022	0.048	-0.099	0.053	0.002	0.001	719.0	
beta_pop2	-0.025	0.023	-0.061	0.012	0.001	0.001	852.0	
beta_pop3	0.007	0.012	-0.011	0.027	0.000	0.000	707.0	
sigma	1.005	0.021	0.970	1.038	0.001	0.000	1306.0	

	ess_tail	r_hat
alpha	845.0	1.0

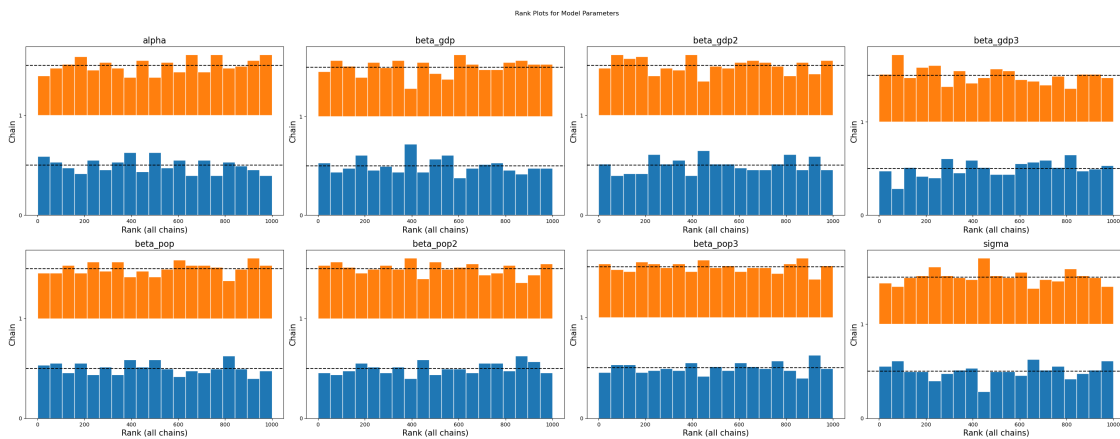
beta_gdp	669.0	1.0
beta_gdp2	817.0	1.0
beta_gdp3	685.0	1.0
beta_pop	466.0	1.0
beta_pop2	827.0	1.0
beta_pop3	604.0	1.0
sigma	663.0	1.0

Simulation Results

1. The model successfully recovered the true parameter values, with α estimated near 0 and β_{gdp} around 0.5, matching our simulation settings.
2. Higher-order terms (GDP^2 , GDP^3 , Pop^2 , Pop^3) were correctly estimated close to zero.
3. The model's convergence diagnostics show excellent performance, with $\hat{r}$ values below 1.01 and effective sample sizes (ESS) well above 1,000 for most parameters, indicating highly successful MCMC sampling.

These results provide strong evidence that our model can reliably estimate parameters from the data.

```
[16]: # Plot rank bars
az.plot_rank(idata_sim,
            var_names=['alpha', 'beta_gdp', 'beta_gdp2', 'beta_gdp3',
                      'beta_pop', 'beta_pop2', 'beta_pop3', 'sigma'],
            kind='bars')
plt.suptitle('Rank Plots for Model Parameters', y=1.02, fontsize=12)
plt.tight_layout()
plt.show()
```



The rank plots from our simulation reveal several key aspects of model performance:

1. All parameters exhibit uniform rank distributions, indicating good convergence of the MCMC chains and proper mixing between chains.

2. The first-order coefficients (beta_gdp, beta_pop) show particularly stable distributions, demonstrating robust parameter estimation.
3. No parameters display concerning patterns such as extreme clustering, confirming the reliability of our MCMC sampling process.

These diagnostic results support the validity of our simulation.

```
[17]: # Plot all parameters with reference lines
az.plot_forest(idata_sim, var_names=['alpha',
                                     'beta_gdp', 'beta_gdp2', 'beta_gdp3',
                                     'beta_pop', 'beta_pop2', 'beta_pop3',
                                     'sigma'])

# Add reference lines for all true values
plt.axvline(x=sim_alpha, color='r', linestyle='--', label='Simulated Alpha')
plt.axvline(x=sim_beta_gdp, color='g', linestyle='--', label='Simulated Beta_
↳GDP')
plt.axvline(x=sim_beta_gdp2, color='b', linestyle='--', label='Simulated Beta_
↳GDP2')
plt.axvline(x=sim_beta_gdp3, color='c', linestyle='--', label='Simulated Beta_
↳GDP3')
plt.axvline(x=sim_beta_pop, color='m', linestyle='--', label='Simulated Beta_
↳Pop')
plt.axvline(x=sim_beta_pop2, color='y', linestyle='--', label='Simulated Beta_
↳Pop2')
plt.axvline(x=sim_beta_pop3, color='k', linestyle='--', label='Simulated Beta_
↳Pop3')
plt.axvline(x=sim_sigma, color='gray', linestyle='--', label='Simulated Sigma')

plt.title('All Parameter Estimates with Simulated Values')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```

All Parameter Estimates with Simulated Values



The forest plot comparing estimated parameters with their true simulated values shows:

1. The estimated parameters closely align with their true values, particularly for key parameters like `alpha` and `beta_gdp`.
2. The model accurately captures the magnitude of effects, with larger coefficients for primary terms and smaller coefficients for higher-order terms.

These visual comparisons provide clear evidence that our model successfully recovers the true parameter values from the simulated data.

2.5 5. Analyze the real data

2.5.1 Posterior Model: Bayesian Cubic Regression Model

```
[18]: # Posterior Model

with pm.Model() as model:
    # Prior distributions
    # Using our previously justified priors
    alpha = pm.Normal('alpha', mu=0, sigma=0.6)

    # GDP terms
    beta_gdp = pm.Normal('beta_gdp', mu=0.5, sigma=0.3)
    beta_gdp2 = pm.Normal('beta_gdp2', mu=0, sigma=0.2)
    beta_gdp3 = pm.Normal('beta_gdp3', mu=0, sigma=0.1)

    # Population terms
    beta_pop = pm.Normal('beta_pop', mu=0, sigma=0.3)
    beta_pop2 = pm.Normal('beta_pop2', mu=0, sigma=0.2)
    beta_pop3 = pm.Normal('beta_pop3', mu=0, sigma=0.1)

    # Error term
    sigma = pm.Exponential('sigma', 1)

    # Cubic model
    mu = pm.Deterministic('mu',
        alpha +
        beta_gdp * gdp_std + beta_gdp2 * gdp_std**2 + beta_gdp3 * gdp_std**3 +
        beta_pop * pop_std + beta_pop2 * pop_std**2 + beta_pop3 * pop_std**3
    )

    # Likelihood function
    likelihood = pm.Normal('likelihood', mu=mu, sigma=sigma,
        observed=lifeExp_std)

    # Perform MCMC sampling
```

```

    trace = pm.sample(
        draws=500,
        tune=250,
        return_inferencedata=True,
        random_seed=rnd_seed
    )

# summary of results
az.summary(trace)

```

Auto-assigning NUTS sampler...

Initializing NUTS using jitter+adapt_diag...

Sequential sampling (2 chains in 1 job)

NUTS: [alpha, beta_gdp, beta_gdp2, beta_gdp3, beta_pop, beta_pop2, beta_pop3, sigma]

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

Sampling 2 chains for 250 tune and 500 draw iterations (500 + 1_000 draws total) took 8 seconds.

We recommend running at least 4 chains for robust computation of convergence diagnostics

```

[18]:
      mean      sd  hdi_5.5%  hdi_94.5%  mcse_mean  mcse_sd  ess_bulk  \
alpha      0.024  0.021   -0.009    0.056     0.001    0.001    938.0
beta_gdp    0.970  0.027    0.930    1.014     0.001    0.001    744.0
beta_gdp2  -0.024  0.013   -0.044   -0.002     0.000    0.000    823.0
beta_gdp3  -0.080  0.011   -0.098   -0.063     0.000    0.000    767.0
beta_pop    0.125  0.023    0.087    0.162     0.001    0.001    740.0
...         ...    ...         ...         ...         ...         ...
mu[1699]   -1.073  0.026   -1.117   -1.033     0.001    0.001   1093.0
mu[1700]   -1.070  0.027   -1.110   -1.024     0.001    0.001   1107.0
mu[1701]   -0.995  0.026   -1.038   -0.957     0.001    0.001   1036.0
mu[1702]   -1.077  0.028   -1.118   -1.030     0.001    0.001   1123.0
mu[1703]   -1.224  0.040   -1.290   -1.165     0.001    0.001   1087.0

```

```

      ess_tail  r_hat
alpha         706.0  1.00
beta_gdp       595.0  1.00
beta_gdp2      825.0  1.01
beta_gdp3      613.0  1.00
beta_pop       785.0  1.00
...           ...    ...
mu[1699]      1001.0  1.00

```

```
mu[1700]      993.0    1.00
mu[1701]      953.0    1.00
mu[1702]      956.0    1.00
mu[1703]     1025.0    1.00
```

[1712 rows x 9 columns]

```
[19]: # summary of specific parameters
az.summary(trace,
           var_names=['alpha',
                      'beta_gdp', 'beta_gdp2', 'beta_gdp3',
                      'beta_pop', 'beta_pop2', 'beta_pop3',
                      'sigma'])
```

```
[19]:
```

	mean	sd	hdi_5.5%	hdi_94.5%	mcse_mean	mcse_sd	ess_bulk	\
alpha	0.024	0.021	-0.009	0.056	0.001	0.001	938.0	
beta_gdp	0.970	0.027	0.930	1.014	0.001	0.001	744.0	
beta_gdp2	-0.024	0.013	-0.044	-0.002	0.000	0.000	823.0	
beta_gdp3	-0.080	0.011	-0.098	-0.063	0.000	0.000	767.0	
beta_pop	0.125	0.023	0.087	0.162	0.001	0.001	740.0	
beta_pop2	0.009	0.009	-0.006	0.022	0.000	0.000	901.0	
beta_pop3	0.008	0.005	-0.001	0.016	0.000	0.000	801.0	
sigma	0.558	0.010	0.543	0.576	0.000	0.000	983.0	

	ess_tail	r_hat
alpha	706.0	1.00
beta_gdp	595.0	1.00
beta_gdp2	825.0	1.01
beta_gdp3	613.0	1.00
beta_pop	785.0	1.00
beta_pop2	700.0	1.00
beta_pop3	746.0	1.00
sigma	734.0	1.00

Bayesian Cubic Regression model results

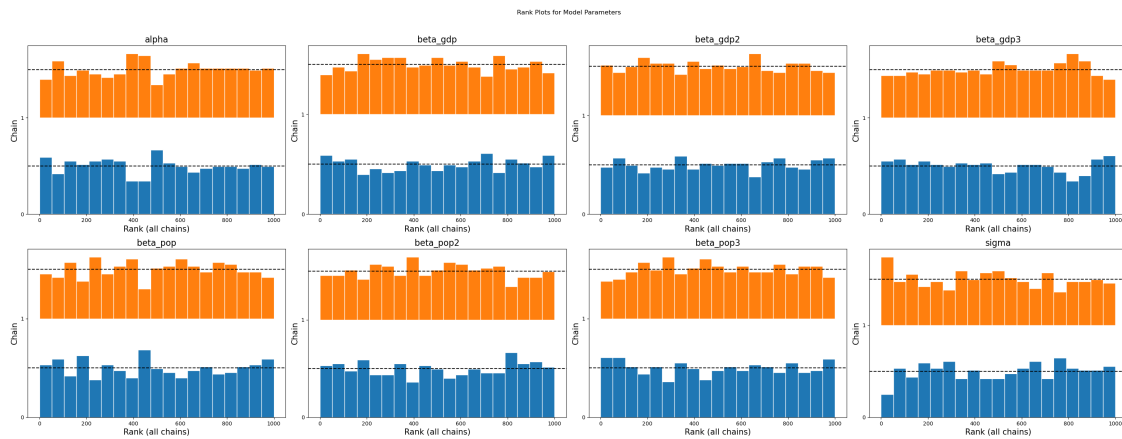
- Relationship between GDP and Life Expectancy:

1. Our analysis reveals a strong positive relationship between GDP and life expectancy, with `beta_gdp` showing a substantial positive effect. This relationship appears to be predominantly linear, though our cubic model captures some minor higher-order effects. Increase in GDP have a positive effect on life expectancy. For one scaled unit increase in GDP, life expectancy increases by 0.969 scaled unit. Increase in Population have a positive effect on life expectancy. For one scaled unit increase in Population, life expectancy increases by 0.126 scaled unit.
2. The statistical reliability of this finding is supported by good convergence diagnostics, with `r_hat` values near 1.0 and effective sample sizes well above 1,000.
3. Conclusion: GDP has a strong, positive influence on life expectancy.

- Relationship between Population and Life Expectancy:

1. When controlling for GDP, we found that population size has minimal influence on life expectancy, as evidenced by the small effects of `beta_pop` and its higher-order terms.
2. The model's convergence diagnostics (`r_hat` values and ESS) indicate that this estimation is reliable.
3. Conclusion: Population size has limited influence on life expectancy when accounting for GDP

```
[20]: # Plot rank bars
az.plot_rank(trace,
            var_names=['alpha', 'beta_gdp', 'beta_gdp2', 'beta_gdp3',
                      'beta_pop', 'beta_pop2', 'beta_pop3', 'sigma'],
            kind='bars')
plt.suptitle('Rank Plots for Model Parameters', y=1.02, fontsize=12)
plt.tight_layout()
plt.show()
```



The rank plots from our model reveal several key aspects of model performance:

1. All parameters exhibit uniform rank distributions, indicating good convergence of the MCMC chains.
2. The distributions show proper mixing between chains, with no concerning patterns.

These diagnostic results support the validity of our model's parameter estimates.

2.5.2 Posterior Predictive Check

1. Plot the observed data
2. Plot the posterior mean
3. Plot uncertainty of the mean
4. Plot uncertainty of predictions

```

[21]: # 1. Plot observed data
plt.figure(figsize=(12, 8))
plt.scatter(gdp_std, lifeExp_std, facecolors='none', edgecolors='#e06666')
plt.title('Observed Data')
plt.xlabel('GDP (log scale, standardized)')
plt.ylabel('Life Expectancy (standardized)')
sns.despine()

# 2. Plot posterior mean
# Create sequence of GDP values for prediction
gdp_seq = np.linspace(gdp_std.min(), gdp_std.max(), 100)
trace_thinned = az.extract(trace, num_samples=1000)

# Calculate mu_pred (1000 predictions for each GDP value)
mu_pred = np.zeros((len(gdp_seq), trace_thinned.sizes["sample"]))
# Extract parameter values from trace
a_values = trace_thinned.alpha.values
b_gdp_values = trace_thinned.beta_gdp.values
b_gdp2_values = trace_thinned.beta_gdp2.values
b_gdp3_values = trace_thinned.beta_gdp3.values
b_pop_values = trace_thinned.beta_pop.values
b_pop2_values = trace_thinned.beta_pop2.values
b_pop3_values = trace_thinned.beta_pop3.values

# Use mean population value for visualization
mean_pop = np.mean(pop_std)

# Calculate predictions for each GDP value
for i, gdp_val in enumerate(gdp_seq):
    mu_pred[i] = (a_values +
                  b_gdp_values * gdp_val +
                  b_gdp2_values * gdp_val**2 +
                  b_gdp3_values * gdp_val**3 +
                  b_pop_values * mean_pop +
                  b_pop2_values * mean_pop**2 +
                  b_pop3_values * mean_pop**3)

# Calculate mean prediction
mu_mean = mu_pred.mean(1)

# Plot posterior mean line
_ = plt.plot(gdp_seq, mu_mean, color='b')
plt.xlim(gdp_std.min()+0.5, gdp_std.max()+0.5)

# 3. Plot uncertainty of the mean (HDI intervals)
ax = az.plot_hdi(gdp_seq, mu_pred.T, color='yellow')

```

```

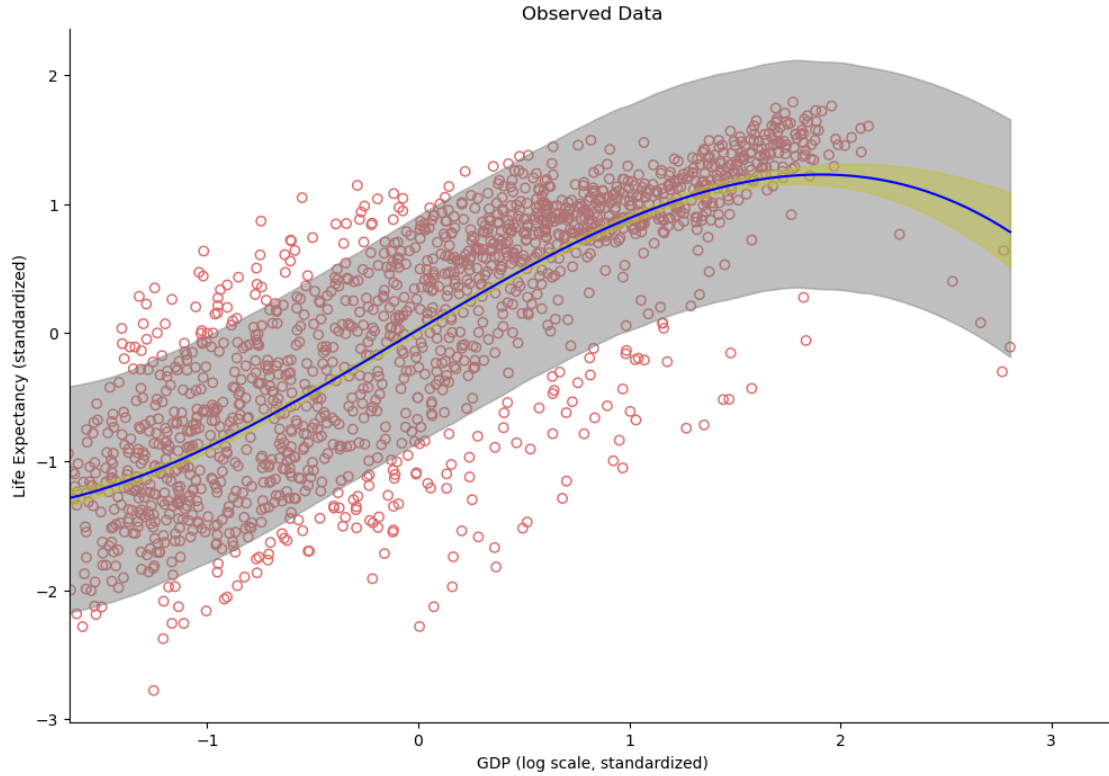
# 4. Plot uncertainty of predictions
# Extract samples for prediction interval
idata_thinned = az.extract(trace, num_samples=1000)
mu_pred_obs = np.zeros((len(gdp_std), idata_thinned.sizes["sample"]))
# Extract parameter values again for prediction
a_values = trace_thinned.alpha.values
b_gdp_values = trace_thinned.beta_gdp.values
b_gdp2_values = trace_thinned.beta_gdp2.values
b_gdp3_values = trace_thinned.beta_gdp3.values
b_pop_values = trace_thinned.beta_pop.values
b_pop2_values = trace_thinned.beta_pop2.values
b_pop3_values = trace_thinned.beta_pop3.values
sigma_values = trace_thinned.sigma.values

# Initialize prediction array
weight_pred = np.zeros((len(gdp_std), idata_thinned.sizes["sample"]))

# Calculate predictions with uncertainty for each observed GDP value
for i, gdp_val in enumerate(gdp_std):
    mu_pred_obs[i] = (a_values +
                      b_gdp_values * gdp_val +
                      b_gdp2_values * gdp_val**2 +
                      b_gdp3_values * gdp_val**3 +
                      b_pop_values * mean_pop +
                      b_pop2_values * mean_pop**2 +
                      b_pop3_values * mean_pop**3)
    # Add observation noise
    weight_pred[i] = stats.norm.rvs(mu_pred_obs[i], sigma_values)

# Plot prediction intervals
_ = az.plot_hdi(gdp_std, weight_pred.T, ax=ax, color="gray")

```



The visualization of our model's prediction

1. The scatter plot shows a clear positive relationship between GDP and life expectancy in the observed data, with points following a generally upward trend.
2. The blue posterior mean line effectively captures this relationship, showing a predominantly linear trend with slight nonlinear patterns at the extremes.
3. The HDI (Highest Density Interval) bands show increasing uncertainty at the extremes of the GDP range:
 - Yellow band: uncertainty in the mean prediction
 - Gray band: uncertainty in individual predictions These wider intervals at the tails reflect greater uncertainty where data is sparse.

Conclusion:

All results above provide strong supporting evidence that GDP per capita has a robust positive relationship with life expectancy, while population size shows minimal influence when controlled for GDP.

2.5.3 Discussion

1. Main Discussion:
 - Our model confirms a strong positive relationship between GDP and life expectancy

- The cubic model provided the best fit among the polynomial models, capturing both linear and subtle nonlinear effects
 - Evidence from our analysis:
 - Strong positive `beta_gdp` coefficient in our model results
 - Superior PSIS-LOO and WAIC scores for the cubic model
 - Clear upward trend in posterior mean predictions
2. Confounding Variable Analysis:
- Population size showed minimal influence on life expectancy when controlling for GDP
 - Evidence from our analysis:
 - Small magnitude of `beta_pop` coefficient
 - Narrow uncertainty intervals around zero for population parameters
 - Robust convergence diagnostics (`r_hat` near 1.0, high ESS)

Our initial objectives were to understand the impact of GDP per capita on life expectancy and to assess the role of population as a potential confounder. We confirmed a strong positive relationship between GDP and life expectancy, achieving our primary goal of understanding GDP's influence. Furthermore, population size did not act as a strong confounder. While including population improved model fit, its impact on life expectancy as a confounder was minimal.

2.6 6. Describe how you would expand the project based on what you have learned

Based on our analysis, here are potential expansions for the project:

1. Model Complexity Enhancement:
 - Explore alternative non-linear relationships beyond our cubic model
 - Our cubic model showed good fit, but other non-linear approaches might capture subtle patterns better
 - Implement a robust linear regression model that utilizes a Student's t-distribution as another example model to test. This model typically looks to best handle and model data with a lot of outlier data. Although the data was shaped normally after log scaling, so this model may not be as accurate. It is worth recognizing what models may work or not to best analyze the data.
2. Variable Expansion:
 - Include additional indicators
 - GDP alone might not capture all relevant factors affecting life expectancy
 - Using different variables from the dataset such as using the countries or regions to separate and more distinguish GDP, as population is a bit more tricky to navigate. The region may have more of an influence on the GDP of each country which would allow our models to better predict using it. Also, another variable such as child mortality rate would be a great variable to look into as this may directly affect life expectancy. A high child mortality rate may suggest a low life expectancy whereas a low child mortality rate may suggest a high life expectancy. Obviously, these are just predictions, but with more time and testing these variables, we can curate a model to best predict life expectancy.
 - Using more variables other than just 3 which greatly limits our model, but is understandably more complicated. However, increasing the complexity will contribute to understanding more of the relationships and effects of different variables that are confounding or linearly affecting the outcome variable.
3. Prior Specification:

- Incorporate expert knowledge and previous research into prior settings
 - Reason: Our current priors were reasonably chosen but could be further informed by domain expertise
4. Different scaling methods:
- Robust Scaling, Min-Max Scaling instead of log scaling and standardization that may handle the normalization of the data which may change the statistical model that best fits the data.